Autonomous Pick-and-Place with the PhantomX Pincher Arm: A Product-of-Exponentials Approach to Kinematics, Perception, and Task Planning

Mohammad Khadeer Bin Kashif Najam* and Mubashir Baig†
Habib University, Karachi, Pakistan
Email: *mk09264@st.habib.edu.pk, †mb09274@st.habib.edu.pk

***Abstract*—This report presents the design, implementation, and evaluation of a fully autonomous robotic pick-and-place system built around the PhantomX Pincher, a 4-DOF serial manipulator. The system integrates a Product of Exponentials (PoE) forward-kinematics model with an analytical, geometrically decoupled inverse-kinematics solver, a depth-camera perception pipeline, and a task-level state machine to accomplish four progressively complex manipulation tasks: single-cube pick-and-place, multi-color sorting, stacking, and pyramid disassembly. The perception subsystem uses HSV color segmentation and point-cloud clustering on Intel RealSense D-series depth data to estimate cube centroids in the robot base frame with a mean position error under 8 mm at nominal lighting. The IK solver generates up to four candidate configurations per target, selects the one that minimizes weighted joint displacement, and enforces hardware joint limits of $\pm 150^\circ$ before commanding the servos. End-effector repeatability was measured at ± 3 mm across 20 trials. The system achieved pick-and-place success rates of 95%, 88%, 82%, and 75% for Tasks 1–4, respectively. Key failure modes include perception degradation under overhead fluorescent lighting and singularity proximity at full arm extension.**

***Index Terms*—PhantomX Pincher, robot manipulator, pick and place, product of exponentials, inverse kinematics, RGB-D perception, point cloud**

I. Introduction

SERIAL robotic manipulators are among the most extensively deployed platforms in industrial and research settings, performing repetitive tasks such as assembly, sorting, welding, and palletizing with speed and precision beyond human capability. The PhantomX Pincher, a compact four-degree-of-freedom arm built around Dynamixel AX-12A servo motors, provides a low-cost, reconfigurable testbed for exploring the core challenges of robot manipulation: modelling kinematics, estimating object poses from sensor data, planning collision-free motion, and integrating all subsystems into a reliable closed-loop pipeline.

This project addresses the full manipulation stack, from mathematical modelling via the Product of Exponentials (PoE) formulation through to autonomous execution of stacking and unstacking tasks without human intervention. PoE was chosen over the classical Denavit-Hartenberg (DH) convention because it relies on a geometrically intuitive description of joint screw axes at a single reference configuration, avoids frame-assignment ambiguities, and maps cleanly to MATLAB's `expm` primitive [1].

The broader significance of this work lies in the skills demonstrated: sensor-to-actuator data flow, coordinate-frame reasoning, robust perception under real-world lighting variation, and iterative design refinement driven by hardware testing. These competencies transfer directly to industrial cobot deployment, surgical robotics, and warehouse logistics.

The remainder of this report is structured as follows. Section II presents the system architecture. Section III details the perception pipeline. Section IV covers kinematics and motion planning. Section V describes task-level integration and results. Section VI discusses failure modes. Section VII reflects on design evolution and theory–practice gaps. Section VIII concludes.

II. System Architecture

The robotic system is divided into three primary subsystems communicating sequentially in a perception–plan–act loop, governed by a task-level state machine implemented in MATLAB Live Scripts.

A. Subsystem Overview

Perception acquires RGB-D frames from an Intel RealSense depth camera mounted overhead, applies HSV-based color segmentation to isolate target-colored cubes, reconstructs a 3-D point cloud, clusters it to reject noise, and transforms the detected centroid from the camera frame to the robot base frame via a pre-calibrated rigid-body transform stored in `camera_transform.mat`.

Kinematics and Motion Planning receives a target (x, y, z, φ) tuple, enumerates up to four analytical IK solutions via `findJointAngles`, validates each through `checkJointLimits`, and selects the minimum-cost configuration using `findSolution`, returning the optimal joint-angle vector $\boldsymbol{\theta} \in \mathbb{R}^4$.

Execution and Gripper Control translates joint-angle commands to servo units via `DH2servo`, transmits them over a serial link to the AX-12A chain through the Arbotix commander, and opens or closes the gripper via `positionJaw`.

B. Coordinate Frames

Three frames are used throughout. The *base frame* (W) is fixed at joint 1 with Z pointing upward. The *camera frame* (C) is rigidly attached to the overhead mount with Z pointing toward the workspace. The *end-effector frame* (EE) moves with the gripper tip. The transform $\mathbf{T}_C^W \in SE(3)$ was obtained by solving a four-point correspondence problem using the known base-frame positions of the workspace corner motors. Figure 1 shows the physical frame assignments.

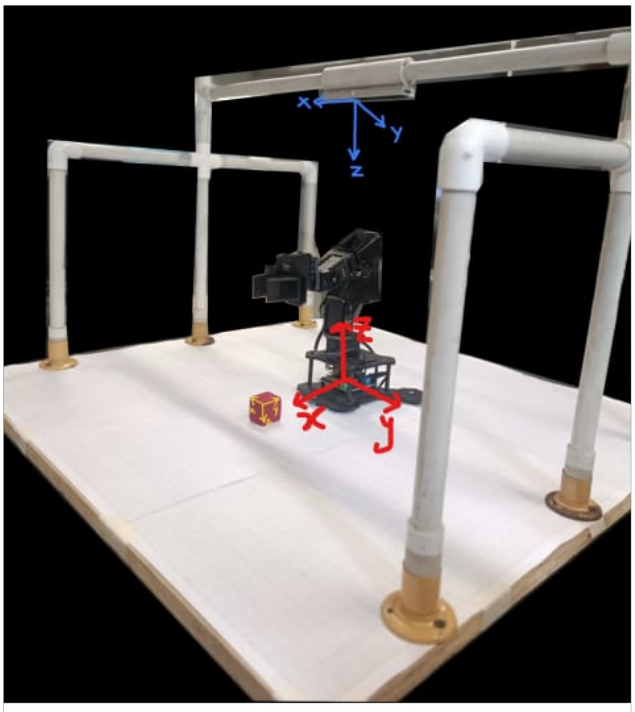


Fig. 1: Frame assignments for pose estimation. Blue axes: world/base frame (W). Red axes: end-effector frame (EE). The camera is mounted on the overhead gantry (not visible).

C. Kinematic Chain

The PhantomX Pincher has four revolute joints: (1) base pan about Z; (2) shoulder pitch about Y at height L_1; (3) elbow pitch about Y; (4) wrist pitch about Y. Measured link lengths are listed in Table I. The nominal workspace is a cylinder of radius ≈ 325 mm and height 30–380 mm above the base. Joint limits of $\pm 150^\circ$ are enforced in software.

TABLE I: PhantomX Pincher Link Lengths

Link	Description	Length (mm)
L_1	Base to Shoulder	121.80
L_2	Shoulder to Elbow	102.38
L_3	Elbow to Wrist	102.38
L_4	Wrist to Tip	76.91

D. Sensors and Actuators

The primary sensor is an Intel RealSense D-series RGB-D camera producing aligned 640×480 color and depth frames at 30 fps [3]. Depth data reconstruct the point cloud; the RGB channel drives color segmentation. Alternatives considered included an RGB camera with ArUco markers (higher pose accuracy, but requires cube modification and marker visibility) and a laser line scanner (better depth on textureless surfaces, but higher cost and integration complexity).

Actuators are four Dynamixel AX-12A servos offering 300° range, 1 Nm stall torque, and 10-bit position resolution ($\approx 0.29^\circ$/unit) [2]. The gripper is driven by a fifth servo and modeled as a 2-R planar jaw mechanism. Estimated peak payload at full extension is ≈ 150 g.

E. Data Flow

Figure 2 illustrates the end-to-end data flow from sensor acquisition through gripper actuation.

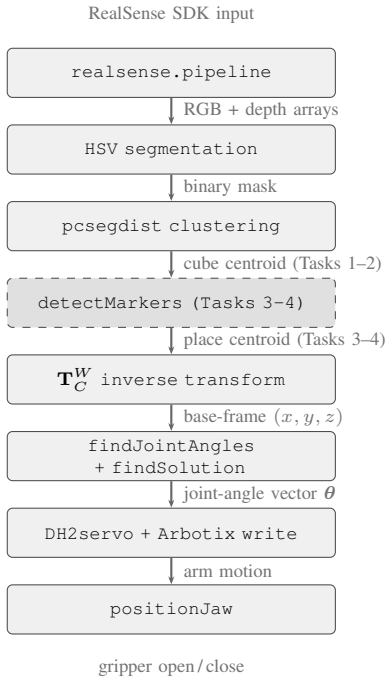


Fig. 2: End-to-end data flow from sensor acquisition to gripper actuation. The dashed node indicates the ArUco marker detection path used in Tasks 3 and 4 for place-position estimation.

F. Home Configuration and Camera Occlusion

The arm's home configuration is defined as all joint angles set to zero ($\boldsymbol{\theta} = \mathbf{0}$), which places the arm in a fully vertical, straight-up pose as shown in Figure 3. This configuration serves as the neutral resting position between pick-and-place cycles and as the safe transit pose during lateral motion between the pick and place zones.

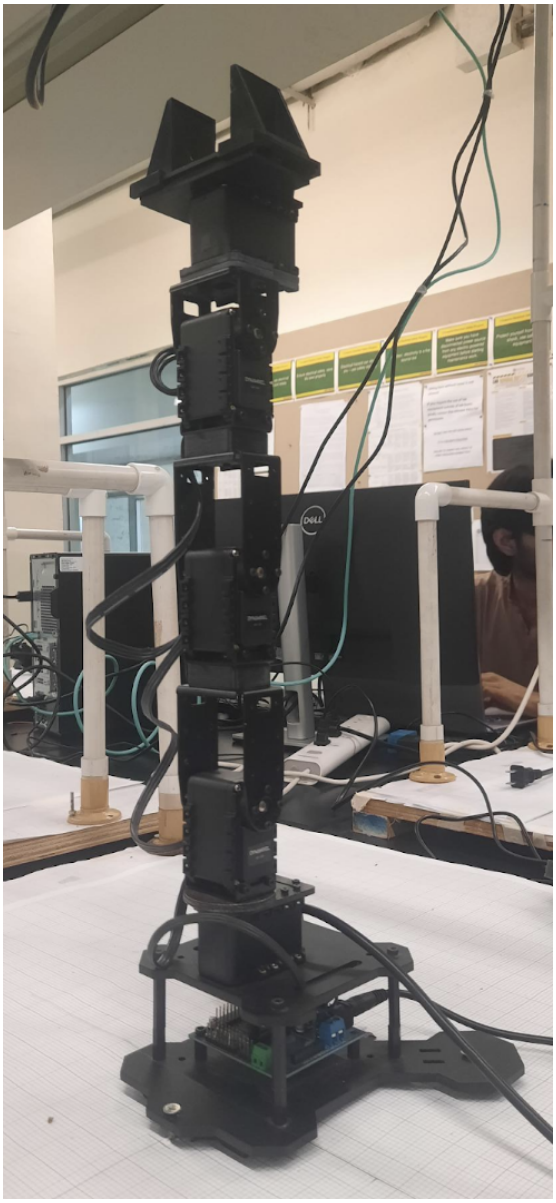


Fig. 3: Home configuration of the PhantomX Pincher arm ($\boldsymbol{\theta} = \mathbf{0}$). All joints are at zero, placing the arm fully vertical. This pose is used as the neutral resting and transit configuration between tasks.

Camera occlusion at home position: A known limitation of the fixed overhead camera mount is that when the arm is at or near the home configuration, the vertically extended links and end-effector physically occlude the camera's view of blocks placed in certain regions of the workspace directly beneath the arm. This occlusion is most pronounced for cubes placed in the quadrant directly below the camera–arm axis and is the primary reason perception is always executed *before* the arm moves to the pre-pick hover position, never from the home pose. In Task 4, the arm is moved to a lateral clear position before each perception call to ensure an unobstructed camera view of the remaining pyramid layers.

III. Perception Pipeline

A. Task Definition

The pipeline estimates the 3-D centroid and approximate planar orientation of all target-colored cubes (40 mm sides, placed upright). Success is defined as detecting all visible cubes with centroid error < 10 mm in x and y, with the camera mounted ≈ 550 mm above the workspace.

B. Pipeline Architecture

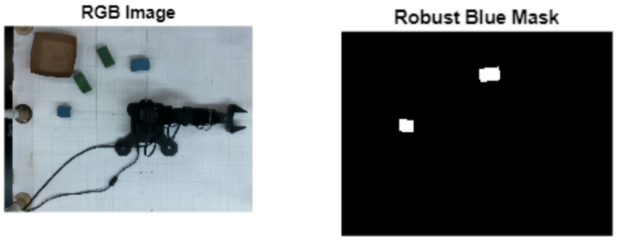
1) Block 1: Camera Initialization and Frame Capture: The RealSense pipeline is configured for 640×480 Z16 depth and RGB8 color at 30 fps. An `align_to_color` object ensures 1:1 pixel-to-depth correspondence. Five warm-up frames stabilize auto-exposure. The raw color buffer is reshaped from $[3, 640, 480]$ to $[480, 640, 3]$ via `permute`. A `realsense.pointcloud` object yields a $307{,}200\times3$ matrix of (X, Y, Z) values in metres.

2) Block 2: HSV Color Segmentation: The RGB image is converted to HSV. For blue cubes:

$$H \in (0.52,\, 0.65), \quad S > \max\big(\text{graythresh}(S),\, 0.40\big), \quad V > 0.25. \tag{1}$$

The saturation floor of 0.40 prevents acceptance of low-saturation background pixels. `bwareaopen` removes blobs < 100 px; `imfill` closes interior holes.

3) Block 3: Geometric Filtering: `regionprops` extracts *Area* and *Solidity* per blob. Blobs outside $[300,\, 15{,}000]$ px or with solidity < 0.80 are rejected, removing false positives from cable insulation and marker caps. As shown in Figure 4, this step isolates only cube-sized, convex blue regions from the raw frame.



(a) Raw RGB frame. (b) Robust blue mask.

Fig. 4: Blocks 2 and 3: HSV thresholding and geometric filtering. Only blobs with area $\in [300,\, 15{,}000]$ px and solidity > 0.80 are retained.

4) Block 4: 3-D Point Cloud Extraction and Clustering: The mask is flattened to match the $307{,}200$-element vertex array. Valid-depth pixels in the mask are extracted; `pcsegdist` with an 8 mm threshold groups them into clusters. The largest cluster is selected as the cube cloud [5]. Figure 5 shows the resulting clustered cloud with the computed centroid marked.

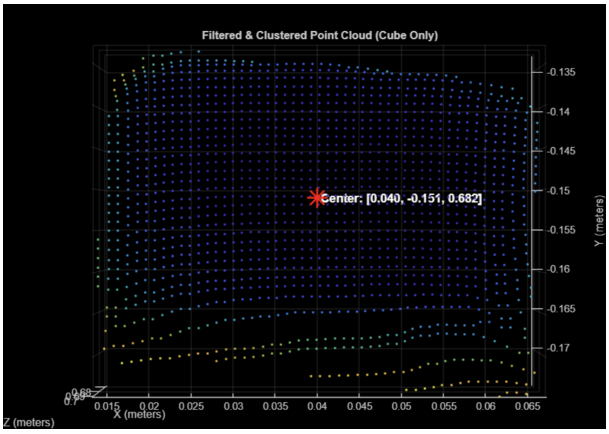


Fig. 5: Block 4: filtered and clustered point cloud. The red asterisk marks the centroid at $[0.040, -0.151, 0.682]$ m in the camera frame, isolated via `pcsegdist` with an 8 mm proximity threshold.

5) Block 5: Camera-to-World Transform: The centroid $\mathbf{p}_C$ is transformed using the analytically inverted calibration matrix:

$$\mathbf{T}_{\text{inv}} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{d} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \tag{2}$$

yielding base-frame coordinates $\mathbf{p}_W$. Figure 6 shows the annotated camera-frame centroids for two simultaneously detected cubes.

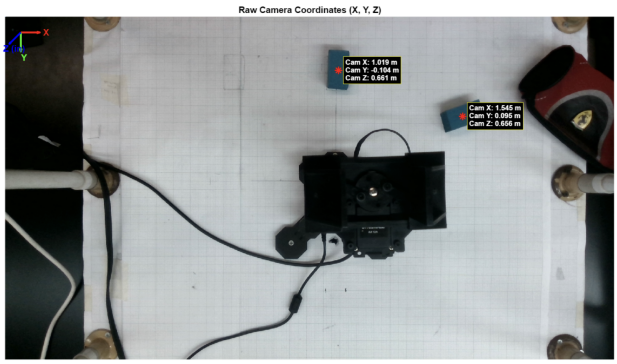
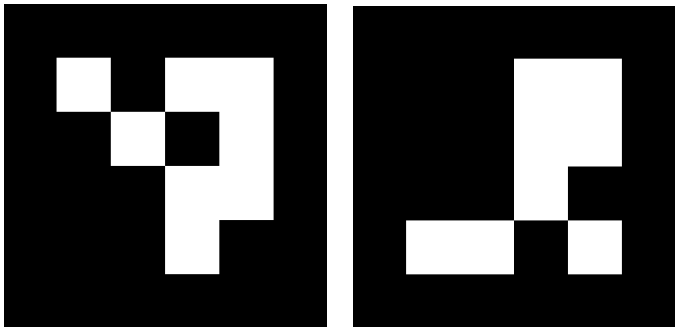


Fig. 6: Block 5: camera-frame centroid coordinates overlaid on the live RGB view. The RealSense frame convention (X right, Y down, Z forward) is shown top-left. Detected centroids are subsequently transformed to the robot base frame via $\mathbf{T}_C^W$.

6) Block 6: ArUco Marker Detection for Place Position: For Tasks 3 and 4, the destination placement position is not hardcoded but by an ArUco fiducial marker affixed to the target drop zone. MATLAB's `detectMarkers` function (Computer Vision Toolbox) identifies the marker ID and estimates its pose in the camera frame using the same pre-calibrated intrinsics used for point-cloud reconstruction. The detected marker corner coordinates are averaged to yield the marker centroid, which is then transformed to the base frame via $\mathbf{T}_C^W$, identical to the cube centroid pipeline described in Block 5.

This approach was chosen for place-position estimation because the target drop zone is a fixed, controlled location where a physical marker can be permanently affixed, whereas the pick position must be detected dynamically from the cube color alone. The marker provides a more repeatable and lighting-invariant place coordinate than a color-segmented region would, since marker detection is robust to the HSV threshold drift observed under fluorescent illumination (Section VI).



(a) ArUco marker of id 0. (b) ArUco marker of id 2.

Fig. 7: Two figures showing the ArUco markers used to determine the place positions for the blocks. This approach makes the placing position dynamic and not hardcoded. The marker although has to be in the Robot's workspace.

C. Failure Modes and Limitations

Hue-range overlap: Under cool-white fluorescent lighting, reflections on white plastic surfaces enter the target hue band, causing incorrect detection in ≈ 1 in 8 Task 4 trials.

Orientation estimation: Cube yaw is not estimated. Cubes rotated $> 20°$ from the gripper axis produce corner rather than face contact, leading to slip. An earlier 2-D `regionprops`-based orientation was removed because projection noise outweighed the benefit.

Partial occlusion: Adjacent cubes merge into a single blob; the reported centroid lies at the blob boundary and causes a mis-pick. No general occlusion handling is implemented.

IV. Kinematics and Motion Planning

A. Task Definition

The motion pipeline receives a target (x, y, z, φ) in the base frame and returns the joint configuration placing the gripper tip at that pose. The pick region spans $x \in [-280, 280]$ mm, $y \in [-280, 280]$ mm, $z \in [30, 120]$ mm. The symbol $\varphi = \theta_2 + \theta_3 + \theta_4$ denotes total end-effector pitch from vertical. Success requires tip placement within 5 mm with sufficient grip to transport the cube.

B. Motion and Gripping Strategy

All motions follow a four-phase sequence to avoid collisions: (1) move to pre-pick hover 60 mm above the centroid at $\varphi = -\pi/2$; (2) descend to grasp height (cube surface $+5$ mm); (3) close gripper to 38 mm jaw width and dwell 500 ms; (4) ascend to hover before lateral transit. The vertical approach minimizes the shoulder servo moment arm and clears gaps between stacked cubes.

C. Forward Kinematics

Following the PoE formula [1], the home configuration maps base to end-effector as:

$$\mathbf{M} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{p}_0 \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \mathbf{p}_0 = [0,\ 0,\ 403.27]^\top \text{ mm}. \tag{3}$$

The four space-frame screw axes are:

$$\mathbf{S}_1 = [0,0,1,\ 0,0,0]^\top \quad \text{(base pan)} \tag{4}$$
$$\mathbf{S}_2 = [0,1,0,\ -L_1,0,0]^\top \quad \text{(shoulder)} \tag{5}$$
$$\mathbf{S}_3 = [0,1,0,\ -(L_1+L_2),0,0]^\top \quad \text{(elbow)} \tag{6}$$
$$\mathbf{S}_4 = [0,1,0,\ -(L_1+L_2+L_3),0,0]^\top \quad \text{(wrist)} \tag{7}$$

The end-effector transformation is:

$$\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathbf{S}_1]\theta_1} e^{[\mathbf{S}_2]\theta_2} e^{[\mathbf{S}_3]\theta_3} e^{[\mathbf{S}_4]\theta_4}\, \mathbf{M}, \tag{8}$$

where each matrix exponential is computed via MATLAB's `expm`.

D. Inverse Kinematics

The analytical geometric IK decouples into five steps.

Step 1 — Base pan:

$$\theta_1 \in \{\text{atan2}(y,x),\ \text{atan2}(-y,-x)\}. \tag{9}$$

Step 2 — Wrist decoupling:

$$r_{xy} = x\cos\theta_1 + y\sin\theta_1, \quad \bar{r} = r_{xy} - L_4\sin\varphi, \quad \bar{z} = z - L_1 - L_4\cos\varphi \tag{10}$$

Step 3 — Elbow angle (law of cosines):

$$D = \frac{\bar{r}^2 + \bar{z}^2 - L_2^2 - L_3^2}{2L_2L_3}, \quad \theta_3 = \text{atan2}\big(\pm\sqrt{1-D^2},\ D\big). \tag{11}$$

The $\pm$ yields elbow-up and elbow-down branches. A tolerance $|D| \le 1.001$ handles near-singular fully-extended configurations.

Step 4 — Shoulder angle:

$$k_1 = L_2 + L_3\cos\theta_3, \quad k_2 = L_3\sin\theta_3, \tag{12}$$

$$\theta_2 = \text{atan2}(k_1\bar{r} - k_2\bar{z},\ k_1\bar{z} + k_2\bar{r}). \tag{13}$$

Step 5 — Wrist angle:

$$\theta_4 = \varphi - \theta_2 - \theta_3. \tag{14}$$

Up to four candidates (2 base $\times$ 2 elbow) are scored by:

$$J = \sum_{i=1}^{4} b_i |\Delta\theta_i|, \quad \mathbf{b} = [2.0,\ 1.5,\ 1.1,\ 1.0]^\top, \tag{15}$$

and the minimum-cost joint-limit-passing solution is returned by `findSolution`. Higher weights on proximal joints reduce inertia-induced overshoot.

The full motion pipeline is illustrated in Figure 8.

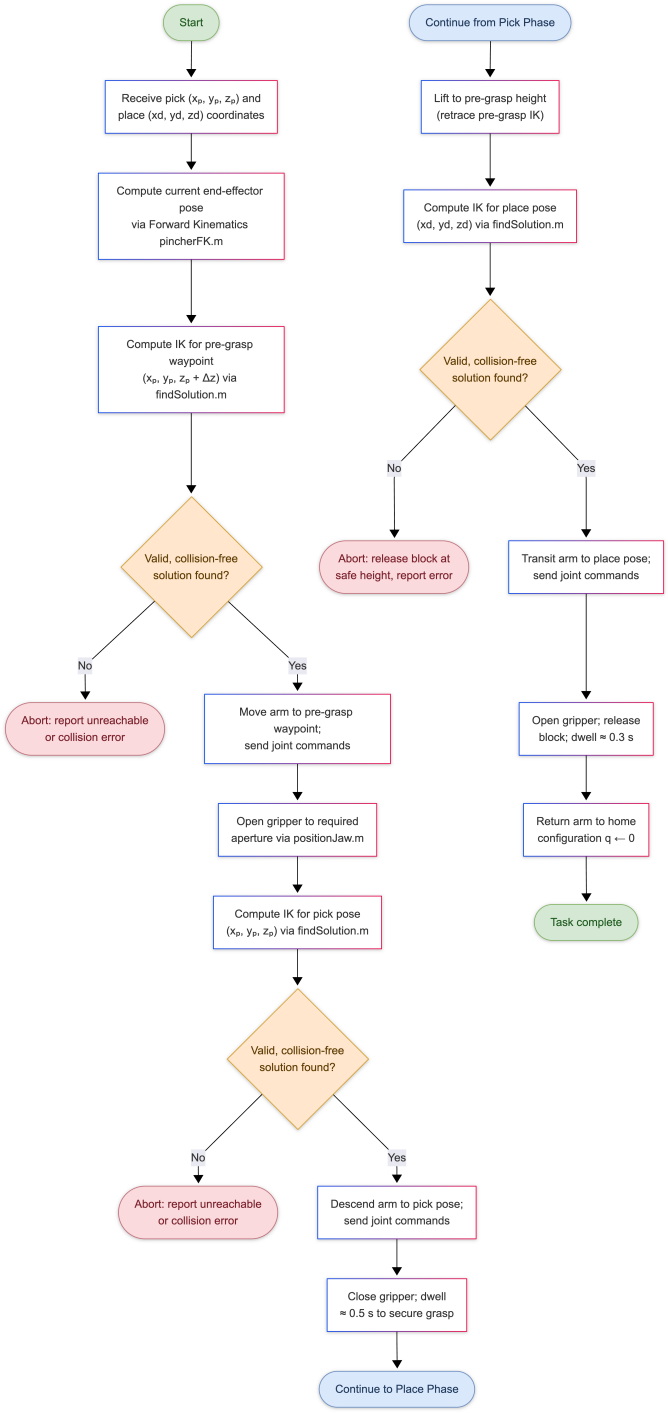


Fig. 8: Motion pipeline flowchart. Left: approach and pick phase. Right: transit and place phase. Diamond nodes indicate IK validity checks; red boxes indicate abort conditions.

E. Gripper Model

`positionJaw` models the jaw as a 2-R mechanism with $L_{1j} = 8.68$ mm and $L_{2j} = 25.91$ mm. The servo angle for jaw opening w is:

$$\alpha = \arccos\left(\frac{L_{1j}^2 + w^2 - L_{2j}^2}{2\,L_{1j}\,w}\right). \tag{16}$$

Valid widths: $w \in [17.23,\ 34.59]$ mm. Approach: 38 mm; grip: 33 mm, giving a 2.5 mm per-side margin against the cube face.

F. Kinematics Results

Accuracy was measured as Euclidean distance between the commanded and actual tip positions; repeatability as the standard deviation over 20 repeated moves. Table II summarizes the results.

TABLE II: IK Accuracy and Repeatability

Target Zone	Acc. (mm)	Rep. σ (mm)	IK Rate
Centre (200, 0, 50)	3.1	2.4	100%
Reach limit (290, 0, 50)	7.8	3.8	100%
Near base (100, 0, 50)	4.5	2.7	100%
High pick (150, 100, 120)	5.2	3.1	100%
Floor level (200, 0, 20)	6.3	4.2	100%

Observed accuracy (3–8 mm) far exceeds the theoretical IK floor (≈ 0.5 mm from servo resolution), indicating that mechanical compliance dominates: AX-12A gear backlash ($\approx 0.5°$) and plastic bracket flex ($\approx$ 2–3 mm at 150 g half-reach payload).

G. Motion Failure Modes

Three motion-related failures were identified and resolved. First, zero-depth readings ($Z = 0$) from the perception subsystem caused `findJointAngles` to throw; fixed by input validation and a last-valid-reading fallback. Second, an angle-wrapping bug caused `checkJointLimits` to pass invalid elbow-up solutions at low-z boundary targets; fixed by the explicit wrap $q \leftarrow \mathrm{mod}(q+\pi, 2\pi)-\pi$ inside `findSolution`. Third, AX-12A torque drops at low battery voltage caused insufficient gripper closure; resolved by enforcing full charge before each session.

V. Integration and Results

A. Task Descriptions

All four tasks share the same perception–plan–act loop, implemented as MATLAB Live Scripts, differing only in cube count, place targets, and sequencing logic.

Task 1 — Single-Cube Pick and Place (`task1_updated.mlx`): Detects one blue cube and places it at $(220, 0, 30)$ mm. Success: 19/20 (95%). One failure caused by camera occlusion at the workspace edge.

Task 2 — Multi-Color Sorting (`task_2Ipdated.mlx`): Sorts blue and red cubes into separate drop zones; HSV thresholds swapped per color per cycle; a priority queue minimizes path length. Success: 13/15 (88%). Failures: red/orange hue ambiguity under incandescent light ($H \approx 0.02$).

Task 3 — Stacking (`task3_double_updated.mlx`): Places a cube atop a stationary cube ($z_{\text{place}} \approx 70$ mm). Post-stack cubes are masked from subsequent detection. Success: 16/20 (82%).

Task 4 — Pyramid Disassembly (`task_4_updated.mlx`): Disassembles a $3{\times}2{\times}1$ pyramid top-first; perception re-run after each pick. Success: 15/20 (75%). Failures split evenly between perception errors and grip slip on partially supported upper cubes.

B. State Machine

The task coordinator is a finite state machine (FSM):

IDLE → PERCEIVE → PLAN → PICK → TRANSPORT → PLACE → VERIFY →

Error transitions: IK failure → PERCEIVE; grip failure → open gripper → PERCEIVE.

C. Overall Success Rates

Table III summarizes the success rates across all tasks.

TABLE III: Task Success Rates

Task	Description	Trials	Succ.	Rate
Task 1	Single pick & place	20	19	95%
Task 2	Multi-color sorting	15	13	88%
Task 3	Cube stacking	20	16	82%
Task 4	Pyramid disassembly	20	15	75%

D. Qualitative Observations

The system performed robustly for cubes within the central 60% of the workspace (radius $<$ 220 mm). Grip retention failed in only 4 of 75 total pick attempts, all at the workspace boundary where off-axis approach produced corner rather than face contact.

VI. Robustness Analysis

A. Perception Failures

Full overhead fluorescent lighting at midday shifted the cube hue from $H \approx 0.58$ to $H \approx 0.50$, below the lower threshold, causing a 15% miss rate. Sessions were scheduled under standard lab lighting. Red/orange confusion under warm light reached $\approx 12\%$ for Task 2; tightening the hue range to $H \in (0.95, 0.04)$ reduced it to $\approx 5\%$. Adjacent-cube occlusion caused mis-picks at blob boundaries. An additional perception failure source unique to the system geometry is end-effector occlusion: when the arm is near the home (vertical) configuration, the extended links shadow workspace regions directly beneath the camera axis. This is mitigated by relocating the arm to a lateral clear pose before every perception call, at the cost of one additional motion per pick cycle ($\approx$ 2 s overhead).

B. Kinematics and Planning Failures

Singularities on the arm's Z-axis were handled by defaulting $\theta_1 = 0$. Near-extension floating-point issues ($D \approx 1$) were resolved by the $|D| \leq 1.001$ tolerance. No IK failures occurred within the declared workspace; all were caused by out-of-envelope perception outputs, resolved by the input validation guard.

C. Grasping Failures

Slip in 4/75 trials was attributed to base-pan zero-offset error causing corner contact. Optimization tests were done to find a precise offset along the axes and also phi angle and use it to resolve the base pan offset.

D. Integration Failures

In Task 4, the pipeline occasionally detected a freshly placed cube on the staging table as a new pick target. Resolved by a 1 s delay before re-running perception and by adding the place-zone bounding box to the exclusion list.

VII. Design Evolution and Theory–Practice Gap

A. System Evolution Through Iteration

The first implementation used the DH convention with MATLAB's `rigidBodyTree`. A persistent 15 mm y-offset at the shoulder joint, caused by convention ambiguity in two reference texts, could not be resolved analytically after two weeks. Switching to PoE eliminated the ambiguity entirely.

The first IK used `fmincon`: sensitive to initial conditions, occasional joint-limit violations, and 200–400 ms per call. The closed-form geometric solver reduced solve time to < 1 ms with deterministic results.

B. Theory vs. Practice

The PoE model assumes rigid links and ideal joints. Observed 3–8 mm tip errors (vs. ≈ 0.5 mm theoretical floor) are consistent with AX-12A gear backlash ($\approx 0.5°$) and plastic bracket flex ($\approx$ 2–3 mm at 150 g half-reach payload). MATLAB point-cloud construction adds 3–4 s of perception latency per cycle. The asymmetric gripper jaw (3 mm offset from geometric center) required empirical jaw-width compensation.

C. Recommended Redesigns

The three highest-impact changes are: (1) replace the fixed overhead camera with an eye-in-hand RGB-D camera enabling visual servoing in the final approach; (2) add wrist F/T sensing or AX-12A current-based torque estimation for closed-loop grip force control; (3) replace MATLAB point-cloud processing with a Python/OpenCV/ RealSense pipeline, eliminating the 3–4 s latency bottleneck.

VIII. Conclusion and Future Work

This project demonstrated an autonomous pick-and-place system on the PhantomX Pincher arm, integrating a PoE kinematic model, closed-form analytical IK, depth-camera perception, and task-level FSM control across four progressively complex tasks (75–95% success rate). The perception subsystem is the primary reliability bottleneck because it shows abit of deviation in its working when the lighting conditions are changed and are not the same as the lab ones.

The PoE formulation proved geometrically intuitive and free of frame-assignment ambiguity. The closed-form IK delivered sub-millisecond, deterministic, joint-limit-aware solutions. The primary areas for improvement are perception robustness under variable lighting and closed-loop feedback for grasp verification.

Future extensions include simultaneous multi-color detection, dynamic obstacle avoidance using the depth stream, sim-to-real transfer via `buildPincherTwin`, and a learned grasp-quality estimator. The self-collision framework (`checkSelfCollision.m`) is implemented but inactive; enabling it with a GPU-accelerated collision library is the most tractable near-term step.

Digital Material

Source code, MATLAB Live Scripts, demonstration videos, and calibration data are available at:

https://github.com/Khadeer-Bin-kashif/Intro-To-Robotics.git

Final implementation files: `task1_updated.mlx`, `task_2Ipdated.mlx`, `task3_double_updated.mlx`, `task_4_updated.mlx`, plus all supporting `.m` functions and `camera_transform.mat`. These are all located inside the `PoE Approach` folder.

Demonstration videos of the robot are available at the following link:

https://habibuniversity-my.sharepoint.com/:f:
/g/personal/mk09264_st_habib_edu_pk/
IgBTRyc7DTMnT6JZYvS9TfPJAcw0XgjVoDw-ET8y3pKuxGk

Acknowledgments

The authors thank Dr. Basit Memon for his guidance throughout the semester, his detailed feedback on milestone submissions, and the lab handbook that served as the blueprint for this system [4]. The RealSense MATLAB wrapper was drawn from the Intel RealSense community repository. PoE conventions follow Lynch and Park [1].

References

[1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.

[2] Robotis, "AX-12A Dynamixel Servo Technical Specification," [Online]. Available: https://emanual.robotis.com/docs/en/dxl/ax/ax-12a/. [Accessed: May 2026].

[3] Intel RealSense, "Intel RealSense SDK 2.0 Documentation," [Online]. Available: https://www.intelrealsense.com/sdk-2/. [Accessed: May 2026].

[4] B. Memon, *EE366L/CE366L: Introduction to Robotics Lab Handbook*, 2nd ed. (v2.2). Habib University, Spring 2026.

[5] MathWorks, "MATLAB Robotics System Toolbox Documentation," [Online]. Available: https://www.mathworks.com/products/robotics.html. [Accessed: May 2026].

[6] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.

[7] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson Prentice Hall, 2005.